Audience: LEWG, SG14
Document number: P0447R4
Date: 2017-10-16
Project: Introduction of std::colony to the standard library
Reply-to: Matthew Bentley <mattreecebentley@gmail.com>

Introduction of std::colony to the standard library

Table of Contents

I. Introduction

In a human colony, people move into houses then when they move on those houses are filled by other people (assuming limited resources). New houses are not built unless necessary, as it's inefficient to do so. But as the colony grows, we'd start building larger and larger houses for efficiency's sake - if a small one became empty it could be demolished and the land cleared to make way for a larger one. Lastly, if we wanted to find people, but had many of empty houses in a row, we'd want to avoid checking each and every individual house. For that reason we might want to use a numbering system to allow us to skip over rows of empty houses.

That metaphor roughly describes the three core aspects of a colony container - erased-element location reuse upon reinsertion, block-based allocation with a growth factor, and jump-counting skipfields. Memory locations of erased elements are marked as empty and 'recycled', with new elements utilizing them upon future insertion, meanwhile being skipped over during iteration. This avoids reallocation upon erasure. New memory blocks are not allocated unless necessary - and when a memory block becomes empty, it is freed to the OS (under certain circumstances it may be retained for performance reasons - see clear() and erase()). This avoids needless iteration over erased blocks and is one part of enabling O(1) iteration time complexity. The other part is the use of what I've called a jump-counting skipfield, which allows contiguous erased elements to be skipped during iteration and also increases iteration performance compared to a boolean skipfield. Lastly, elements in a colony are allocated in multiple memory blocks, with a growth factor, enabling pointers to elements to stay valid after insertions and reducing the total number of allocations.

Colony is designed to be a higher-performance container for situations where the ratio of insertions/erasures to iterations is relatively high. It allows only for unordered insertion but is sortable, has fast iteration and gives the

programmer direct pointer/iterator access to elements without the threat of those pointers/iterators being invalidated by subsequent insertions or erasure. This makes it useful for object-oriented scenarios and many other frameworks such as entity component systems. A colony has some similarities to a ``bag'' container, but without id lookups or element counting. Instead of using id's as lookups for individual elements, the insert function returns an iterator which is valid for the lifetime of the element. The iterator can of course be dereferenced to obtain a pointer to the element, which is also valid for the element's lifetime. Since direct pointer access or iterator access does not have the overhead of id lookup, this is another positive point for colony, performance-wise. Based on the reference implementation (plf::colony) it has been established that colonies have better overall performance than standard library containers or container modifications when the following are valid:

a. Insertion order is unimportant

b. Insertions and erasures to the container occur frequently in performance-critical code, ***and***

c. Pointers/iterators to non-erased container elements may not be invalidated by insertion or erasure.

The performance characteristics compared to other containers, for the most-commonly-used operations in the above circumstance are:

- *Singular (non-fill, unordered) insertion*: better than any std:: library container.
- *Erasure (from random location)*: better than any std:: library container.
- *Iteration*: better than any std:: library container capable of preserving pointer/iterator validity post-erasure/insertion (eg. map, multiset, list). Some variance here for scalar types and different CPUs. Where pointer/iterator validity is unimportant, better than any std:: library container except deque and vector.

There are some vector/deque modifications/workarounds which can outperform colony for iteration while maintaining link validity, but which have slower insertion and erasure speeds, and typically also have a cost to usability or memory requirements. Under scenarios involving high levels of modification colony will outperform these as well. These results, along with performance comparisons to the unmodified std:: containers, are explored in detail in the Appendix B Benchmarks.

In terms of usage, there are two access patterns for stored elements; the first is to simply iterate over the colony and process each element (or skip some using the advance/prev/next functions). The second is to store the iterator returned by the insert() function (or a pointer derived from the iterator) in some other structure and access the inserted element that way. This is particularly useful in object-oriented code, as we shall see.

II. Motivation and Scope

Note: Throughout this document I will use the term 'link' to denote any form of referencing between elements whether it be via iterators/pointers/indexes/references/id's.

There are situations where data is heavily interlinked, iterated over frequently, and changing often. An example is a typical video game engine. Most games will have a central 'entity' class, regardless of their overall schema. These are 'has a'-style objects rather than 'is a'-style objects, which reference other shared resources like sprites, sounds and suchforth. These resources are typically located in separate containers/arrays. The entities are in turn referenced by other structures such as quadtrees/octrees, level structures and so on.

The entities may be erased at any time (for example, a wall gets destroyed and no longer requires processing) and new entities may be inserted (for example, a new enemy is created). All the while, inter-linkages between entities, resources and superstructures such as levels and quadtrees, are required to stay valid in order for the game to run. The order of the entities and resources themselves within the containers is, in the context of a game, typically unimportant.

What is needed is a container (or modified use of a container) which allows for insertions and erasures to occur without invalidating links between elements within containers. Unfortunately the container with the best iteration performance in the standard library, vector[1], also loses pointer validity to elements within it upon insertion, and also pointer/index validity upon erasure. This tends to lead to sophisticated, and often restrictive, workarounds when developers attempt to utilize vector or similar containers under the above circumstances.

As another example, non-game particle simulation (weather, physics etcetera) involve large clusters of particles which interact with external objects and each other, where the particles each have individual properties (spin, momentum, direction etc) and are being created and destroyed continuously. The order of the particles is unimportant, but what is important is the speed of erasure and insertion, since this is a high-modification scenario. No standard library container has both strong insertion and non-back erasure speed, and again this is a good match for colony.

Here are some more specific requirements with regards to game engines, verified by game developers within SG14:

a. Elements within data collections refer to elements within other data collections (through a variety of methods - indices, pointers, etc). These references must stay valid throughout the course of the game/level. Any container which causes pointer or index invalidation creates difficulties or necessitates workarounds.

b. Order is unimportant for the most part. The majority of data is simply iterated over, transformed, referred to and utilized with no regard to order.

c. Erasing or otherwise "deactivating" objects occurs frequently in performance-critical code. For this reason methods of erasure which create strong performance penalties are avoided.

d. Inserting new objects in performance-critical code (during gameplay) is also common - for example, a tree drops leaves, or a player spawns in an online multiplayer game.

e. It is not always clear in advance how many elements there will be in a container at the beginning of development, or at the beginning of a level during play. Genericized game engines in particular have to adapt to considerably different user requirements and scopes. For this reason extensible containers which can expand and contract in realtime are necessary.

f. Due to the effects of cache on performance, memory storage which is more-or-less contiguous is preferred.

g. Memory waste is avoided.

std::vector in it's default state does not meet these requirements due to:

1. Poor (non-fill) singular insertion performance (regardless of insertion position) due to the need for reallocation upon reaching capacity
2. Insert invalidates pointers/iterators to all elements
3. Erase invalidates pointers/iterators/indexes to all elements afer the erased element

Game developers therefore either develop custom solutions for each scenario or implement workarounds for vector. The most common workarounds are most likely the following or derivatives:

1. Using a boolean flag or similar to indicate the inactivity of an object (as opposed to actually erasing from the vector). Elements flagged as inactive are skipped during iteration.

   Advantages: Fast "deactivation". Easy to manage in multi-access environments.
   Disadvantages: Slower to iterate due to branching.

2. Using a vector of data and a secondary vector of indexes. When erasing, the erasure occurs only in the vector of indexes, not the vector of data. When iterating it iterates over the vector of indexes and accesses the data from the vector of data via the remaining indexes.

   Advantages: Fast iteration.
   Disadvantages: Erasure still incurs some reallocation cost which can increase jitter.

3. Combining a swap-and-pop approach to erasure with some form of dereferenced lookup system to enable contiguous element iteration (sometimes called a 'packed array': http://bitsquid.blogspot.ca/2011/09/managing-decoupling-part-4-id-lookup.html).
   Advantages: Iteration is at standard vector speed.
   Disadvantages: Erasure will be slow if objects are large and/or non-trivially copyable, thereby making swap costs large. All link-based access to elements incur additional costs due to the dereferencing system.

Colony brings a more generic solution to these contexts.

III. Impact On the Standard

This is a pure library addition, no changes necessary to the standard asides from the introduction of the colony container.
A reference implementation of colony is available for download and use: http://www.plflib.org/colony.htm

IV. Design Decisions

As mentioned the key technical features of this container are:

- Unordered non-associative data
- Never invalidates pointers/iterators to non-erased elements (iterators pointing to end() excluded)
- Reuses or frees memory from erased elements
- Where ratio of insertion/erasure to iteration is high, faster than any std:: library container or container workaround

The abstract requirements needed to support these features are:

1. A multiple-memory-block based allocation pattern which allows for fast removal of memory blocks when they become empty of elements without causing element pointer invalidation within other memory blocks.
2. A skipfield to enable the O(1) skipping over of consecutive erased elements during iteration.
3. A mechanism for reusing erased element locations upon subsequent insertions.

Obviously these three things can be achieved in a variety of different ways. Here's how the reference implementation achieves them:

Memory blocks - chained group allocation pattern

This is essentially a doubly-linked list of nodes (groups) containing (a) memory blocks, (b) memory block metadata and (c) skipfields. The memory blocks themselves have a growth factor of 2. The metadata in question includes information necessary for an iterator to iterate over colony elements, such as the last insertion point within the memory block, and other information useful to specific functions, such as the total number of non-erased elements in the node. This approach keeps the operation of freeing empty memory blocks from the colony structure at O(1) time complexity. Further information is available here: http://www.plflib.org/chained_group_allocation_pattern.htm

A vector of memory blocks would not work in this context as a colony iterator must store a pointer (or index) to it's current group in order for iteration to work, and groups must be removed when empty in order to enable O(1) iteration and avoid cache misses. Erasing a group in a vector would invalidate pointers/indexes to all groups after the erased group, and similarly inserting new groups would invalidate pointers to existing groups. A vector of pointers to dynamically-allocated groups is possible, but likely represents little performance advantage when reallocation costs and structural overhead are taken into consideration. In addition the jitter caused when non-back groups are removed could become problematic.

Skipfield - jump-counting skipfield pattern

This numeric pattern allows for O(1) time complexity iterator operations and fast iteration when compared to boolean skipfields. It stores and modifies (during insertion and erasure) numbers which correspond to the number of elements in runs of consecutive erased elements, and during iteration uses these numbers to skip over the runs. This avoids the looping branch code necessary for iteration with a boolean skipfield - which is slow, but which also cannot be used due to it's O(random) time complexity. It has two variants - a simple and an advanced solution, the latter of which colony uses. The published paper of the advanced version of the jump-counting skipfield pattern is available for viewing here: http://rdcu.be/ty6U,

Iterative performance after having previously erased 75% of all elements in the containers at random, for a std::vector, std::deque and a colony with boolean skipfields, versus a colony with a jump-counting skipfield (GCC 5.1 x64, Intel Core2 processor) - storing a small struct type:

Memory re-use mechanism - reduced_stack

This is a stripped-down custom internal stack class based on plf::stack (http://www.plflib.org/stack.htm), which

outperforms any std:: container in a stack context for all datatypes and across compilers. plf::stack uses the same chained-group allocation pattern as plf::colony. In this context the stack pushes erased element memory locations and these are popped upon subsequent insertions. If a memory block becomes empty and is subsequently removed, the stack must be processed and any memory locations from within the block removed.

Time to push all elements then read-and-pop all elements, plf::stack vs std::stack (std::deque) and std::vector (GCC 5.1 x64) - storing type double:

An alternative approach, using a "free list" (explanation of this concept: http://bitsquid.blogspot.ca/2011/09/managing-decoupling-part-4-id-lookup.html) to record the erased element memory locations, by creating a union between T and T*, has been explored and the following issues identified:

1. Unions in C++ (11 or otherwise) currently do not support the preservation of fundamental functionality of the objects being unioned (n4567, 9.5/2). Copy/move assignment/constructors, constructors and destructors would be lost, which makes this approach untenable.
2. A colony element could be smaller in size than a pointer and thus a union with such would dramatically increase the amount of wasted space in circumstances with low numbers of erasures - moreso if the pointer type supplied by the allocator is non-trivial.
3. When a memory block becomes empty and must be removed as discussed earlier, the free list would need all nodes within that memory block removed. Because a free list will (assuming a non-linear erasure pattern) jump between random memory locations continuously, consolidating the free list would result in an operation filled with cache misses. In the context of jitter-sensitive work this would likely become unacceptable. It might be possible to work around this by using per-memory-block free lists.

V. Technical Specifications

Colony meets the requirements of the C++ Container, AllocatorAwareContainer, and ReversibleContainer concepts.

For the most part the syntax and semantics of colony functions are very similar to current existing std:: C++ libraries. Formal description is as follows:

```
template <class T, class Allocator = std::allocator<T>, typename Skipfield_Type = unsigned short> class colony
```

T - the element type. In general T must meet the requirements of Erasable, CopyAssignable and CopyConstructible. However, if emplace is utilized to insert elements into the colony, and no functions which involve copying or moving are utilized, T is only required to meet the requirements of Erasable. If move-insert is utilized instead of emplace, T must also meet the requirements of MoveConstructible .

`Allocator` - an allocator that is used to acquire memory to store the elements. The type must meet the requirements of Allocator. The behavior is undefined if `Allocator::value_type` is not the same as T.

`Skipfield_Type` - an unsigned integer type. This type is used to form the skipfield which skips over erased T elements. The maximum size of element memory blocks is constrained by this type's bit-depth (due to the nature of a jump-counting skipfield). The default type, `unsigned short`, is 16-bit on most platforms which constrains the size of individual memory blocks to a maximum of 65535 elements. `unsigned short` has been found to be the optimal type for performance based on benchmarking, however making this type user-defined has performance benefits in some scenarios. In the case of small collections (eg. < 512 elements) in a memory-constrained environment, reducing the skipfield bit depth to a Uint8 type will reduce both memory usage and cache saturation without impacting iteration performance.

In the case of very large collections (millions) where memory usage is not a concern and where erasure is less common, one might think that changing the skipfield bitdepth to a larger type may lead to slightly increased iteration performance due to the larger memory block sizes made possible by the largerbit depth and the fewer subsequent block transitions. However in practice the performance benefits appear to be little-to-none, and introduce substantial performance issues if erasures are frequent, as with this approach it takes more erasures for a group to become empty - increasing the frequency of large skips between elements and subsequent cache misses, as well as skipfield update times. From this we can assume it is unlikely for a user to want to use types other than unsigned short and

unsigned char. But since these scenarios are on a per-case basis, it has been considered best to leave control in the hands of the user.

Basic example of usage

```
#include <iostream>
#include "plf_colony.h"

int main(int argc, char **argv)
{
  plf::colony<int> i_colony;

  // Insert 100 ints:
  for (int i = 0; i != 100; ++i)
  {
    i_colony.insert(i);
  }

  // Erase half of them:
  for (plf::colony<int>::iterator it = i_colony.begin(); it != i_colony.end(); ++it)
  {
    it = i_colony.erase(it);
  }

  // Total the remaining ints:
  int total = 0;

  for (plf::colony<int>::iterator it = i_colony.begin(); it != i_colony.end(); ++it)
  {
    total += *it;
  }

  std::cout << "Total: " << total << std::endl;
  std::cin.get();
  return 0;
}
```

Example demonstrating pointer stability

```
#include <iostream>
#include "plf_colony.h"

int main(int argc, char **argv)
{
  plf::colony<int> i_colony;
  plf::colony<int>::iterator it;
  plf::colony<int *> p_colony;
  plf::colony<int *>::iterator p_it;

  // Insert 100 ints to i_colony and pointers to those ints to p_colony:
  for (int i = 0; i != 100; ++i)
  {
    it = i_colony.insert(i);
    p_colony.insert(&(*it));
  }

  // Erase half of the ints:
  for (it = i_colony.begin(); it != i_colony.end(); ++it)
  {
    it = i_colony.erase(it);
  }

  // Erase half of the int pointers:
  for (p_it = p_colony.begin(); p_it != p_colony.end(); ++p_it)
  {
    p_it = p_colony.erase(p_it);
```

```
  }

  // Total the remaining ints via the pointer colony:
  int total = 0;

  for (p_it = p_colony.begin(); p_it != p_colony.end(); ++p_it)
  {
    total += *(*p_it);
  }

  std::cout << "Total: " << total << std::endl;

  if (total == 2500)
  {
    std::cout << "Pointers still valid!" << std::endl;
  }

  std::cin.get();
  return 0;
}
```

Time complexity of main operations

Insert/emplace (single): O(1) amortised unless prior erasures have occurred in the usage lifetime of the colony instance. If prior erasures have occurred, updating the skipfield may require a memmove operation, which creates a variable time complexity depending on the range of skipfield needing to be copied (though in practice this will resolve to a singular raw memory block copy in most scenarios, and the performance impact is negligible). This is O(random) with the range of the random number being between 1 and std::numeric_limits<Skipfield_Type>::max() - 2 (65534 in most instances). Average time complexity varies based on erasure pattern. With a random erasure pattern it will be closer to O(1) amortized.

Insert (multiple): O(N) unless prior erasures have occurred. See Insertion(single) for rules in this case.

Erase (single): If erasures to elements consecutive with the element being erased have not occurred, or only consecutive erasures before the element being erased have occurred, O(1) amortised. If consecutive erasures after the element being erased have occurred, updating of the skipfield requires a memmove operation or vectorized update of O(N) complexity, where n is the number of consecutive erased elements after the element being erased. This is O(random) with the range of the random number being between from 1 and std::numeric_limits<Skipfield_Type>::max() - 2. Average time complexity varies based on erasure pattern, but with a random erasure pattern it's closer to O(1) amortized.

Erase (multiple): ~O(log N)

std::find: O(N)

Iterator operations:
++ and -- : O(1) amortized
begin()/end(): O(1)
advance/next/prev, get_iterator_from_index: between O(1) and O(N), depending on current location, end location and state of colony. Average ~O(log N).

Time complexity of other operations

size(), capacity, max_size(), empty(): O(1)

operator =, operator == and != : O(N)

shrink_to_fit(): O(1)

change_group_sizes, change_minimum_group_size, change_maximum_group_size: O(N)

splice: O(1) amortised

sort: uses std::sort internally so roughly equivalent to the time complexity of the compiler's std::sort implementation

Swap, operator = (move), get_allocator(): O(1)

Operations whose complexity does not have a strong correlation with number of elements:

(*ie. other factors such as trivial destructability of elements more important*)

~colony, clear(), reinitialize(), sort(), get_iterator_from_pointer(), get_index_from_iterator(), get_index_from_reverse_iterator().

Iterator Invalidation

All read-only operations, swap, std::swap, shrink_to_fit	Never
clear, sort, reinitialize, operator =	Always
reserve	Only if capacity is changed
change_group_sizes, change_minimum_group_size, change_maximum_group_size	Only if supplied minimum group size is larger than smallest group in colony, or supplied maximum group size is smaller than largest group in colony.
erase	Only for the erased element
insert, emplace	If an iterator is == end() it may be invalidated by a subsequent insert/emplace. Otherwise it will not be invalidated. Pointers will never be invalidated.

Member types

Member type	**Definition**
`value_type`	`T`
`allocator_type`	`Allocator`
`skipfield_type`	`Skipfield_Type`
`size_type`	`std::allocator_traits<Allocator>::size_type`
`difference_type`	`std::allocator_traits<Allocator>::difference_type`
`reference`	`value_type &`
`const_reference`	`const value_type &`
`pointer`	`std::allocator_traits<Allocator>::pointer`
`const_pointer`	`std::allocator_traits<Allocator>::const_pointer`
`iterator`	`BidirectionalIterator`
`const_iterator`	`Constant BidirectionalIterator`

reverse_iterator	BidirectionalIterator
const_reverse_iterator	Constant BidirectionalIterator

Colony iterators cannot be random access due to the use of a skipfield. This constrains +, -, += or -= operators into being non-O(1). But member overloads for the standard library functions advance(), next(), prev() and distance() are available in the reference implementation, and are significantly faster than O(N) in the majority of scenarios.

A full list of the current reference implementation's constructors and member functions can be found in Appendix A.

VI. Acknowledgements

Thank you's to Glen Fernandes and Ion Gaztanaga for restructuring advice, Robert Ramey for documentation advice, various Boost and SG14 members for support, Baptiste Wicht for teaching me how to construct decent benchmarks, Jonathan Wakely for standards-compliance advice and critiques, Sean Middleditch, Patrice Roy and Guy Davidson for critiques, support and bug reports, Jason Turner and Phil Williams for cross-processor testing, that guy from Lionhead for annoying me enough to get me to get around to implementing the skipfield pattern, Jon Blow for some initial advice and Mike Acton for some influence.

VII. Appendixes

Appendix A - Member functions

Constructors

Default	colony() explicit colony(const allocator_type &alloc)
fill	explicit colony(size_type n, skipfield_type min_group_size = 8, skipfield_type max_group_size = std::numeric_limits<skipfield_type>::max(), const allocator_type &alloc = allocator_type()) explicit colony(size_type n, const value_type &element, skipfield_type min_group_size = 8, skipfield_type max_group_size = std::numeric_limits<skipfield_type>::max(), const allocator_type &alloc = allocator_type())
range	template<typename InputIterator> colony(const InputIterator &first, const InputIterator &last, skipfield_type min_group_size = 8, skipfield_type max_group_size = std::numeric_limits<skipfield_type>::max(), const allocator_type &alloc = allocator_type())
copy	colony(const colony &source) colony(const colony &source, const allocator_type &alloc)
move	colony(colony &&source) noexcept colony(colony &&source, const allocator_type &alloc)
initializer list	colony(const std::initializer_list<value_type> &element_list, skipfield_type min_group_size = 8, skipfield_type max_group_size = std::numeric_limits<skipfield_type>::max(), const allocator_type &alloc = allocator_type())

Some constructor usage examples

- `colony<T> a_colony`

Default constructor - default minimum group size is 8, default maximum group size is std::numeric_limits<skipfield_type>::max() (typically 65535). You cannot set the group sizes from the constructor in this scenario, but you can call the change_group_sizes() member function after construction has occurred.
Example: `plf::colony<int> int_colony;`

- `colony<T, the_allocator<T> > a_colony(const allocator_type &alloc = allocator_type())`

Default constructor, but using a custom memory allocator eg. something other than std::allocator.
Example: `plf::colony<int, tbb::allocator<int> > int_colony;`
Example2:

```
// Using an instance of an allocator as well as it's type
tbb::allocator<int> alloc_instance;
plf::colony<int, tbb::allocator<int> > int_colony(alloc_instance);
```

- `colony<T> colony(size_type n, skipfield_type min_group_size = 8, skipfield_type max_group_size = std::numeric_limits<skipfield_type>::max())`

Fill constructor with value_type unspecified, so the value_type's default constructor is used. n specifies the number of elements to create upon construction. If n is larger than `min_group_size`, the size of the groups created will either be n and `max_group_size`, depending on which is smaller. `min_group_size` (ie. the smallest possible number of elements which can be stored in a colony group) can be defined, as can the `max_group_size`. Setting the group sizes can be a performance advantage if you know in advance roughly how many objects are likely to be stored in your colony long-term - or at least the rough scale of storage. If that case, using this can stop many small initial groups being allocated.
Example: `plf::colony<int> int_colony(62);`

- `colony<T> colony(const std::initializer_list<value_type> &element_list, skipfield_type min_group_size = 8, skipfield_type max_group_size = std::numeric_limits<skipfield_type>::max())`

Using an initialiser list to insert into the colony upon construction.
Example:

```
std::initializer_list<int> &el = {3, 5, 2, 1000};
plf::colony<int> int_colony(el, 64, 512);
```

- `colony<T> a_colony(const colony &source)`

Copy all contents from source colony, removes any empty (erased) element locations in the process. Size of groups created is either the total size of the source colony, or the maximum group size of the source colony, whichever is the smaller.
Example: `plf::colony<int> int_colony_2(int_colony_1);`

- `colony<T> a_colony(colony &&source)`

Move all contents from source colony, does not remove any erased element locations or alter any of the source group sizes. Source colony is now void of contents and can be safely destructed.
Example:

```
plf::colony<int> int_colony_1(50, 5, 512, 512); // Create colony with min and max group sizes set at 512 elements. Fill with 50 instances of int = 5.
plf::colony<int> int_colony_2(std::move(int_colony_1)); // Move all data to int_colony_2. All of the above characteristics are now applied to int_colony2.
```

Iterators

All iterators are bidirectional but also provide >, <, >= and <= for convenience (for example, for comparisons against an end iterator when doing multiple increments within for loops) and within some functions (`distance()` uses these for example). Functions for iterator, reverse_iterator, const_iterator and const_reverse_iterator follow:

```
operator * const
operator -> const noexcept
operator ++
operator --
operator = noexcept
operator == const noexcept
operator != const noexcept
operator < const noexcept
operator > const noexcept
operator <= const noexcept
operator >= const noexcept
```

`base() const (reverse_iterator and const_reverse_iterator only)`

All operators have O(1) amortised time-complexity. Originally there were +=, -=, + and - operators, however the time complexity of these varied from O(N) to O(1) depending on the underlying state of the colony, averaging in at O(log N). As such they were not includable in the iterator functions (as per C++ standards). These have been transplanted to colony's advance(), next(), prev() and distance() member functions. Greater-than/lesser-than operator usage indicates whether an iterator is higher/lower in position compared to another iterator in the same colony (ie. closer to the end/beginning of the colony).

Member functions

Insert

single element	`iterator insert (const value_type &val)`
fill	`iterator insert (size_type n, const value_type &val)`
range	`template <class InputIterator> iterator insert (const InputIterator &first, const InputIterator &last)`
move	`iterator insert (value_type&& val)`
initializer list	`iterator insert (const std::initializer_list<value_type> &il)`

- `iterator insert(const value_type &element)`

  Inserts the element supplied to the colony, using the object's copy-constructor. Will insert the element into a previously erased element slot if one exists, otherwise will insert to back of colony. Returns iterator to location of inserted element. Example:

  ```
  plf::colony<unsigned int> i_colony;
  i_colony.insert(23);
  ```

- `iterator insert(value_type &&element)`

  Moves the element supplied to the colony, using the object's move-constructor. Will insert the element in a previously erased element slot if one exists, otherwise will insert to back of colony. Returns iterator to location of inserted element. Example:

  ```
  std::string string1 = "Some text";

  plf::colony<std::string> data_colony;
  data_colony.insert(std::move(string1));
  ```

- `void insert (const size_type n, const value_type &val)`

  Inserts `n` copies of `val` into the colony. Will insert the element into a previously erased element slot if one exists, otherwise will insert to back of colony. Example:

  ```
  plf::colony<unsigned int> i_colony;
  i_colony.insert(10, 3);
  ```

- `template <class InputIterator> void insert (const InputIterator &first, const InputIterator &last)`

  Inserts a series of `value_type` elements from an external source into a colony holding the same `value_type` (eg. int, float, a particular class, etcetera). Stops inserting once it reaches `last`. Example:

  ```
  // Insert all contents of colony2 into colony1:
  colony1.insert(colony2.begin(), colony2.end());
  ```

- `void insert (const std::initializer_list<value_type> &il)`

  Copies elements from an initializer list into the colony. Will insert the element in a previously erased element slot if one exists, otherwise will insert to back of colony. Example:

```
std::initializer_list<int> some_ints = {4, 3, 2, 5};

plf::colony<int> i_colony;
i_colony.insert(some_ints);
```

Erase

single element	`iterator erase(const_iterator it)`
range	`void erase(const_iterator first, const_iterator last)`

- `iterator erase(const_iterator it)`

  Removes the element pointed to by the supplied iterator, from the colony. Returns an iterator pointing to the next non-erased element in the colony (or to end() if no more elements are available). This must return an iterator because if a colony group becomes entirely empty, it may be removed from the colony, invalidating the existing iterator. A group may either be removed when it becomes empty, or moved to the back of the colony for future insertions and made inactive. The decision to either remove or move should be (largely) implementation-defined, but testing has suggested that the best performance under high-modification occurs when groups are removed unless they meet the maximum group size, or are either of the last two active groups at the back of the colony. Certainly there is no performance advantage to removing the end group, and doing so may introduce edge case performance issues (where multiple insertions/erasures occur frequently and sequentially). The reference implementation currently removes all groups when empty. Example:

```
plf::colony<unsigned int> data_colony(50);
plf::colony<unsigned int>::iterator an_iterator;
an_iterator = data_colony.insert(23);
an_iterator = data_colony.erase(an_iterator);
```

- `void erase(const_iterator first, const_iterator last)`

  Erases all contents of a given colony from `first` to the element before the `last` iterator. The same principles from singular erasure apply to this function regards group retention/removal, etc.Example:

```
plf::colony<int> iterator1 = colony1.begin();
colony1.advance(iterator1, 10);
plf::colony<int> iterator2 = colony1.begin();
colony1.advance(iterator2, 20);
colony1.erase(iterator1, iterator2);
```

Other functions

- `iterator emplace(Arguments &&... parameters)`

  Constructs new element directly within colony. Will insert the element in a previously erased element slot if one exists, otherwise will insert to back of colony. Returns iterator to location of inserted element. "...parameters" are whatever parameters are required by the object's constructor. Example:

```
class simple_class
{
private:
int number;
public:
simple_class(int a_number): number (a_number) {};
};

plf::colony<simple_class> simple_classes;
simple_classes.emplace(45);
```

- `bool empty() const noexcept`

Returns a boolean indicating whether the colony is currently empty of elements.
Example: `if (object_colony.empty()) return;`

- `size_type size() const noexcept`

Returns total number of elements currently stored in container.
Example: `std::cout << i_colony.size() << std::endl;`

- `size_type max_size() const noexcept`

Returns the maximum number of elements that the allocator can store in the container. This is an approximation as it does attempt to measure the memory overhead of the container's internal memory structures. It is not possible to measure the latter because a copy operation may change the number of groups utilized for the same amount of elements, if the maximum or minimum group sizes are different in the source container.
Example: `std::cout << i_colony.max_size() << std::endl;`

- `size_type capacity() const noexcept`

Returns total number of elements currently able to be stored in container without expansion.
Example: `std::cout << i_colony.capacity() << std::endl;`

- `void shrink_to_fit()`

Deallocates any unused groups retained during erase() or unused since the last reserve(). This function cannot invalidate iterators or pointers to elements, nor will it necessarily shrink the capacity to match size().
Example: `i_colony.shrink_to_fit();`

- `void reserve(size_type reserve_amount)`

Preallocates memory space sufficient to store the number of elements indicated by `reserve_amount`. In the implementation the maximum size for this number currently is limited to the maximum group size of the colony and will be truncated if necessary. This restriction will be lifted in a future version. This function is useful from a performance perspective when the user is inserting elements singly, but the overall number of insertions is known in advance. By reserving, colony can forgo creating many smaller memory block allocations (due to colony's growth factor) and reserve a single memory block instead.
Example: `i_colony.reserve(15);`

- `void clear()`

Empties the colony and removes all elements, but retains the empty memory blocks rather than deallocating (current implementation deallocates all memory blocks, but in the future this should be changed to reduce needless reallocation/deallocations).
Example: `object_colony.clear();`

- `void change_group_sizes(const unsigned short min_group_size, const unsigned short max_group_size)`

Changes the minimum and maximum internal group sizes, in terms of number of elements stored per group. If the colony is not empty and either min_group_size is larger than the smallest group in the colony, or max_group_size is smaller than the largest group in the colony, the colony will be internally copy-constructed into a new colony which uses the new group sizes, invalidating all pointers/iterators/references.
Example: `object_colony.change_group_sizes(1000, 10000);`

- `void change_minimum_group_size(const unsigned short min_group_size)`

Changes the minimum internal group size only, in terms of minimum number of elements stored per group. If the colony is not empty and min_group_size is larger than the smallest group in the colony, the colony will be internally copy-constructed into a new colony which uses the new minimum group size, invalidating all pointers/iterators/references.
Example: `object_colony.change_minimum_group_size(100);`

- `void change_maximum_group_size(const unsigned short min_group_size)`

Changes the maximum internal group size only, in terms of maximum number of elements stored per group. If the colony is not empty and either max_group_size is smaller than the largest group in the colony, the colony will be internally copy-constructed into a new colony which uses the new maximum group size, invalidating all pointers/iterators/references.
Example: `object_colony.change_maximum_group_size(1000);`

- `void reinitialize(const unsigned short min_group_size, const unsigned short max_group_size)`

Semantics of function are the same as "clear(); change_group_sizes(min_group_size, max_group_size);", but without the copy-construction code of the change_group_sizes() function - this means it can be used with element types which are non-copy-constructible, unlike change_group_sizes().
Example: `object_colony.reinitialize(1000, 10000);`

- `void splice(colony &source)`

Transfer all elements from source colony into destination colony without invalidating pointers/iterators to either colony's elements (in other words the destination takes ownership of the source's memory blocks). After the splice, the source colony is empty. Splicing is much faster than range-moving or copying all elements from one colony to another. Colony does not guarantee a particular order of elements after splicing, for performance reasons; the insertion location of source elements in the destination colony is chosen based on the most positive performance outcome for subsequent iterations/insertions. For example if the destination colony is {1, 2, 3, 4} and the source colony is {5, 6, 7, 8} the destination colony post-splice could be {1, 2, 3, 4, 5, 6, 7, 8} or {5, 6, 7, 8, 1, 2, 3, 4}, depending on internal data.

Note: if there were many erasures in the destination colony prior to splicing, post-splice insertion and iteration may be marginally slower than if all elements in the source colony had been range-moved/copied to the destination instead (due to the move/copy ability to re-use previously erased element memory locations in the destination). Note2: If the minimum group size of the source is smaller than the destination, the destination will change it's minimum group size to match the source. The same applies for maximum group sizes (if source's is larger, the destination will adjust its).
Example:
```
// Splice two colonies of integers together:
colony<int> colony1 = {1, 2, 3, 4}, colony2 = {5, 6, 7, 8};
colony1.splice(colony2);
```

- `void swap(colony &source) noexcept(std::allocator_traits<the_allocator>::propagate_on_container_swap::value || std::allocator_traits<the_allocator>::is_always_equal::value)`

Swaps the colony's contents with that of source.
Example: `object_colony.swap(other_colony);`

- `void sort();`
  `template <class comparison_function>`
  `void sort(comparison_function compare);`

Sort the content of the colony. By default this compares the colony content using a less-than operator, unless the user supplies a comparison function (ie. same conditions as std::list's sort).
Example:
```
// Sort a colony of integers in ascending order:
int_colony.sort();
// Sort a colony of doubles in descending order:
double_colony.sort(std::greater());
```

- `void splice(colony &source);`

Append data from one colony container to the end of another. This operation removes all elements from the source colony. All pointers/iterators to elements within the source and destination containers retain their validity. (note: not currently implemented in reference implementation - will be in future).
Example:
```
// Append one colony to the end of another:
int_colony.splice(int_colony2);
```

- `colony & operator = (const colony &source)`

Copy the elements from another colony to this colony, clearing this colony of existing elements first.
Example:
```
// Manually swap data_colony1 and data_colony2 in C++03
data_colony3 = data_colony1;
data_colony1 = data_colony2;
data_colony2 = data_colony3;
```

- `colony & operator = (const colony &&source)`

Move the elements from another colony to this colony, clearing this colony of existing elements first. Source colony becomes invalid but can be safely destructed without undefined behaviour.
Example:
```
// Manually swap data_colony1 and data_colony2 in C++11
data_colony3 = std::move(data_colony1);
data_colony1 = std::move(data_colony2);
data_colony2 = std::move(data_colony3);
```

- `bool operator == (const colony &source) const noexcept`

Compare contents of another colony to this colony. Returns a boolean as to whether they are equal.
Example: `if (object_colony == object_colony2) return;`

- `bool operator != (const colony &source) const noexcept`

Compare contents of another colony to this colony. Returns a boolean as to whether they are not equal.
Example: `if (object_colony != object_colony2) return;`

- `iterator begin() const noexcept, iterator end() const noexcept, const_iterator cbegin() const noexcept, const_iterator cend() const noexcept`

Return iterators pointing to, respectively, the first element of the colony and the element one-past the end of the colony.

- `reverse_iterator rbegin() const noexcept, reverse_iterator rend() const noexcept, const_reverse_iterator crbegin() const noexcept, const_reverse_iterator crend() const noexcept`

Return reverse iterators pointing to, respectively, the last element of the colony and the element one-before the first element of the colony. (note: as the reference implementation's crbegin() and rbegin() are derived from --end(), they will throw an exception if the colony is empty of elements and are not noexcept).

- `iterator get_iterator_from_pointer(const element_pointer_type the_pointer) const noexcept`

Getting a pointer from an iterator is simple - simply dereference it then grab the address ie. `"&(*the_iterator);"`. Getting an iterator from a pointer is typically not so simple. This function enables the user to do exactly that. This is expected to be useful in the use-case where external containers are storing pointers to colony elements instead of iterators (as iterators for colonies have 3 times the size of an element pointer) and the program wants to erase the element being pointed to or possibly change the element being pointed to. Converting a pointer to an iterator using this method and then erasing, is about 20% slower on average than erasing when you already have the iterator. This is less dramatic than it sounds, as it is still faster than all other std:: container erasure times. If the function doesn't find a non-erased element within the colony, based on that pointer, it returns `end()`. Otherwise it returns an iterator pointing to the element in question. Example:

```
plf::colony<a_struct> data_colony;
plf::colony<a_struct>::iterator an_iterator;
a_struct struct_instance;
an_iterator = data_colony.insert(struct_instance);
a_struct *struct_pointer = &(*an_iterator);
iterator another_iterator = data_colony.get_iterator_from_pointer(struct_pointer);
if (an_iterator == another_iterator) std::cout << "Iterator is correct" << std::endl;
```

- `template <iterator_type>size_type get_index_from_iterator(iterator_type &the_iterator) const`

While colony is a container with unordered insertion (and is therefore unordered), it still has a (transitory) order which may change upon erasure or insertion. *Temporary* index numbers are therefore obtainable. These can be useful, for example, when creating a saved-game file in a computer game, where certain elements in a container may need to be re-linked to other elements in other container upon reloading the save file. Example:

```
plf::colony<a_struct> data_colony;
plf::colony<a_struct>::iterator an_iterator;
a_struct struct_instance;
data_colony.insert(struct_instance);
data_colony.insert(struct_instance);
an_iterator = data_colony.insert(struct_instance);
unsigned int index = data_colony.get_index_from_iterator(an_iterator);
if (index == 2) std::cout << "Index is correct" << std::endl;
```

- `template <iterator_type>size_type get_index_from_reverse_iterator(const iterator_type &the_iterator) const`

The same as get_index_from_iterator, but for reverse_iterators and const_reverse_iterators. Index is measured from front of colony (same as iterator), not back of colony. Example:

```
plf::colony<a_struct> data_colony;
plf::colony<a_struct>::reverse_iterator r_iterator;
a_struct struct_instance;
data_colony.insert(struct_instance);
```

```
data_colony.insert(struct_instance);
r_iterator = data_colony.rend();
unsigned int index = data_colony.get_index_from_reverse_iterator(r_iterator);
if (index == 1) std::cout << "Index is correct" << std::endl;
```

- `iterator get_iterator_from_index(const size_type index) const`

  As described above, there may be situations where obtaining iterators to specific elements based on an index can be useful, for example, when reloading save files. This function is basically a shorthand to avoid typing "`iterator it = colony.begin(); colony.advance(it, 50);`". Example:

```
plf::colony<a_struct> data_colony;
plf::colony<a_struct>::iterator an_iterator;
a_struct struct_instance;
data_colony.insert(struct_instance);
data_colony.insert(struct_instance);
iterator an_iterator = data_colony.insert(struct_instance);
iterator another_iterator = data_colony.get_iterator_from_index(2);
if (an_iterator == another_iterator) std::cout << "Iterator is correct" << std::endl;
```

- `allocator_type get_allocator() const noexcept`

  Returns a copy of the allocator used by the colony instance.

Non-member functions

Note: these are in fact member functions in the reference implementation in order to avoid an unfixed template bug in MSVC2013 (this bug is not present in any other compiler including MSVC2010/15/17).

- `template <iterator_type> void advance(iterator_type iterator, distance_type distance)`

  Increments/decrements the iterator supplied by the positive or negative amount indicated by *distance*. Speed of incrementation will almost always be faster than using the ++ operator on the iterator for increments greater than 1. In some cases it may be O(1). The iterator_type can be an iterator, const_iterator, reverse_iterator or const_reverse_iterator.
  Example: `colony<int>::iterator it = i_colony.begin();`
  `i_colony.advance(it, 20);`

- `template <iterator_type> iterator_type next(const iterator_type &iterator, distance_type distance)`

  Creates a copy of the iterator supplied, then increments/decrements this iterator by the positive or negative amount indicated by *distance*.
  Example: `colony<int>::iterator it = i_colony.next(i_colony.begin(), 20);`

- `template <iterator_type> iterator_type prev(const iterator_type &iterator, distance_type distance)`

  Creates a copy of the iterator supplied, then decrements/increments this iterator by the positive or negative amount indicated by *distance*.
  Example: `colony<int>::iterator it2 = i_colony.prev(i_colony.end(), 20);`

- `template <iterator_type> difference_type distance(const iterator_type &first, const iterator_type &last) const`

  Measures the distance between two iterators, returning the result, which will be negative if the second iterator supplied is before the first iterator supplied in terms of it's location in the colony. If either iterator is uninitialized, behaviour is undefined.
  Example: `colony<int>::iterator it = i_colony.next(i_colony.begin(), 20);`
  `colony<int>::iterator it2 = i_colony.prev(i_colony.end(), 20);`
  `std::cout "Distance: " i_colony.distance(it, it2) std::endl;`

Non-member functions

- ```
  template <class T, class allocator_type, typename skipfield_type&gt
  void swap (colony<T, allocator_type, skipfield_type> &A, colony<T, allocator_type,
  skipfield_type> &B)
  noexcept(std::allocator_traits<allocator_type>::propagate_on_container_swap::value ||
  std::allocator_traits<the_allocator>::is_always_equal::value)
  ```

  Swaps colony A's contents with that of colony B.
  Example: `swap(object_colony, other_colony);`

Appendix B - reference implementation benchmarks

Benchmark results for colony under GCC 7.1 x64 on an Intel Xeon E3-1241 (Haswell) are here.

Older benchmark results for colony under GCC 5.1 x64 on an Intel E8500 (Core2) are here.

Older benchmark results for colony under MSVC 2015 update 3, on an Intel Xeon E3-1241 (Haswell) are here. There is no commentary for the MSVC results.

Appendix C - Frequently Asked Questions

1. **What are some examples of situations where a colony might improve performance?**

   Some ideal situations to use a colony: cellular/atomic simulation, persistent octtrees/quadtrees, general game entities or destructible-objects in a video game, particle physics, anywhere where objects are being created and destroyed continuously. Also, anywhere where a vector of pointers to dynamically-allocated objects or a std::list would typically end up being used in order to preserve object references but where order is unimportant.

2. **What situations should you explicitly *not* use a colony for?**

   Although in practice the performance difference is small, a colony should not be used as a stack, ie. erasing backwards from the back, and then inserting, then erasing from the back, etcetera. In this case you should use a stack-capable container ie. plf::stack, std::vector or std::deque. The reason is that erasing backwards sequentially creates the greatest time complexity for skipfield updates, as does reinserting to the start of a sequentially-erased skipblock (which is what stack usage will entail). This effect is mitigated somewhat if an entire group is erased, in which case it is released to the OS and subsequent insertions will simply be pushing to back without the need to update a skipfield, but you'd still incur the skipfield update cost during erasure.

3. **If the time complexities of the insert/erase functions are (mostly) O(random, ranged), why are they still fast?**

   The time complexities are largely based on the skipfield updates necessary for the jump-counting skipfield pattern. The skipfield for each group is contiguous and separate from the skipfields for other groups, and so fits into the cache easily (unless the skipfield type is large), thus any changes to it can occur quickly - time complexity is no indicator of performance on a modern CPU for anything less than very large amounts of N (or when the type of N is large). The colony implementation uses `memmove` to modify the skipfield instead of iterative updates for all but one of the insert/erase operations, which decreases performance cost (`memmove` will typically be implemented as a single raw memory chunk copy).

   There is one rarer case in erase which does not use `memmove`, when an element is erased and is surrounded on both sides by consecutive erased elements. In this case it isn't possible to update the skipfield using memmove because the requisite numbers do not exist in the skipfield and therefore cannot be copied, so it is implemented as a vectorized iterative update instead. Again, due to a low amount of branching and the small size of each skipfield the performance impact is minimal.
  ```

4. **Is it similar to a deque?**

A deque is reasonably dissimilar to a colony - being a double-ended queue, it requires a different internal framework. A deque for example can't technically use a linked list of memory blocks because it will make some random_access iterator operations (eg. + operator) non-O(1). A deque and colony have no comparable performance characteristics except for insertion (assuming a good deque implementation). Deque erasure performance varies wildly depending on the implementation compared to std::vector, but is generally similar to vector erasure performance. A deque invalidates pointers to subsequent container elements when erasing elements, which a colony does not.

As described earlier the three core aspects of colony are:

a. A multiple-memory-block based allocation pattern which allows for the removal of memory blocks when they become empty of elements.

b. A skipfield to indicate erasures instead of reallocating elements, the iteration of which should typically not necessitate the use of branching code.

c. A mechanism for recording erased element locations to allow for reuse of erased element memory space upon subsequent insertion.

The only aspect out of these which deque also shares is a multiple-memory-block allocation pattern - not a strong association.

5. **What is colony's Abstract Data Type (ADT)?**

Though I am happy to be proven wrong I suspect colony is it's own abstract data type. While it is similar to a multiset or bag, those utilize key values and are not sortable (by means other than automatically by key value). Colony does not utilize key values, is sortable, and does not provide the sort of functionality one would find in a bag (eg. counting the number of times a specific value occurs). Some have suggested simiarities to deque - see FAQ item above. Similarly if we look at a multiset, an unordered one could be implemented on top of a colony by utilizing a hash table (and would in fact be more efficient than most non-flat implementations). But the actual fact that there is a necessity to add something to make it a multiset (to take and store key values) suggests colony is not an multiset.

6. **What are the thread-safe guarantees?**

Unlike a std::vector, a colony can be read from and written to at the same time (assuming different locations for read and write), however it cannot be iterated over and written to at the same time. If we look at a (non-concurrent implementation of) std::vector's threadsafe matrix to see which basic operations can occur at the same time, it reads as follows (please note push_back() is the same as insertion in this regard):

std::vector	Insertion	Erasure	Iteration	Read
Insertion	No	No	No	No
Erasure	No	No	No	No
Iteration	No	No	Yes	Yes
Read	No	No	Yes	Yes

In other words, multiple reads and iterations over iterators can happen simultaneously, but the potential reallocation and pointer/iterator invalidation caused by insertion/push_back and erasure means those operations cannot occur at the same time as anything else.

Colony on the other hand does not invalidate pointers/iterators to non-erased elements during insertion and erasure, resulting in the following matrix:

plf::colony	Insertion	Erasure	Iteration	Read
Insertion	No	No	No	Yes

Erasure	No	No	No	Mostly*
Iteration	No	No	Yes	Yes
Read	Yes	Mostly*	Yes	Yes

* Erasures will not invalidate iterators unless the iterator points to the erased element.

In other words, reads may occur at the same time as insertions and erasures (provided that the element being erased is not the element being read), multiple reads and iterations may occur at the same time, but iterations may not occur at the same time as an erasure or insertion, as either of these may change the state of the skipfield which's being iterated over. Note that iterators pointing to end() may be invalidated by insertion.

So, colony could be considered more inherently threadsafe than a (non-concurrent implementation of) std::vector, but still has some areas which would require mutexes or atomics to navigate in a multithreaded environment.

For a more fully concurrent version of colony, an atomic boolean skipfield could be utilized instead of a jump-counting one, if one were allowed to ignore time complexity rules for iterator operations. This would enable largely lock-free operation of a colony, at the expense of iteration speed.

7. **Any pitfalls to watch out for?**

    1. Because erased-element memory locations will be reused by `insert()` and `emplace()`, insertion position is essentially random unless no erasures have been made, or an equal number of erasures and insertions have been made.
    2. In the current reference implementation reserve() can only reserve a number of elements up to the maximum number afforded by the skipfield type. Hopefully there will be time to rework colony in future so that it more fully supports reserve.

8. **Am I better off storing iterators or pointers to colony elements?**

    Testing so far indicates that storing pointers and then using `get_iterator_from_pointer()` when or if you need to do an erase operation on the element being pointed to, yields better performance than storing iterators and performing erase directly on the iterator. This is simply due to the size of iterators (3 pointers) in the reference implementation.

9. **Any special-case uses?**

    In the special case where many, many elements are being continually erased/inserted in realtime, you might want to experiment with limiting the size of your internal memory groups in the constructor. The form of this is as follows:
    `plf::vector<object> a_vector;`
    `a_vector.change_group_sizes(500, 5000);`
    where the first number is the minimum size of the internal memory groups and the second is the maximum size. Note these can be the same size, resulting in an unchanging group size for the lifetime of the colony (unless `change_group_sizes` is called again or operator = is called).

    One reason to do this is that it is slightly slower to pop an element location off the internal erased-location-recycling stack, than it is to insert a new element to the end of the colony (the default behaviour when there are no previously-erased elements). If there are any erased elements in the colony, the colony will recycle those memory locations, unless the entire group is empty, at which point it is freed to memory. So if a group size is large and many, many erasures occur but the group is not completely emptied, (a) the number of erased element locations in the recycling stack could get large and increase memory usage and (b) iteration performance may suffer due to large memory gaps between any two non-erased elements. In that scenario you may want to exeriment with benchmarking and limiting the minimum/maximum sizes of the groups, and find the optimal size for a specific use case.

    Please note that the the fill, range and initializer-list constructors can also take group size parameters, making it possible to construct filled colonies using custom group sizes.

10. **Why must groups be removed when empty, or moved to the back of the chain?**

Two reasons:

a. Standards compliance: if groups aren't removed then ++ and - - iterator operations become O(random) in terms of time complexity, making them non-compliant with the C++ standard. At the moment they are O(1) amortised, typically one update for both skipfield and element pointers, but two if a skipfield jump takes the iterator beyond the bounds of the current group and into the next group. But if empty groups are allowed, there could be anywhere between 1 and `size_type` empty groups between the current element and the next. Essentially you get the same scenario as you do when iterating over a boolean skipfield. While it is possible to move these to the back of the colony as trailing groups, and remove their entries from the erased locations stack, this may create performance issues if any of the groups are not at their maximum size (see below).

b. Performance: iterating over empty groups is slower than them not being present, cache wise - but also if you have to allow for empty groups while iterating, then you have to include a while loop in every iteration operation, which increases cache misses and code size. The strategy of removing groups when they become empty also removes (assuming randomized erasure) smaller groups from the colony before larger groups, which has a net result of improving iteration (as with a larger group, more iterations within the group can occur before the end-of-group condition is reached and a transfer to the next group (and subsequent cache miss) occurs). Lastly, pushing to the back of a colony is faster than recycling memory locations as each insertion occurs to a similar memory location and less work is necessary. When a group is removed or moved to the back of the group chain (past end()), it's recyclable memory locations are also removed from memory, hence subsequent insertions are more likely to be pushed to the back of the colony.

11. **Group sizes - what are they based on, how do they expand, etc**

In the reference implementation group sizes start from the either the default minimum size (8 elements, larger if the type stored is small) or an amount defined by the programmer (with a minimum of 3 elements). Subsequent group sizes then increase the *total capacity* of the colony by a factor of 2 (so, 1st group 8 elements, 2nd 8 elements, 3rd 16 elements, 4th 32 elements etcetera) until the maximum group size is reached. The default maximum group size is the maximum possible number that the skipfield bitdepth is capable of representing (std::numeric_limits<skipfield_type>::max()). By default the skipfield type is unsigned short so on most platforms the maximum size of a group would be 65535 elements. Unsigned short (guaranteed to be at least 16 bit, equivalent to C++11's uint_least16_t type) was found to have the best performance in real-world testing due to the balance between memory contiguousness, memory waste and the restriction on skipfield update time complexity. Initially the design also fitted the use-case of gaming better (as games tend to utilize lower numbers of elements than some other fields), and that was the primary development field at the time.

Appendix D - Specific responses to previous committee feedback

1. **Why not 'bag'? Colony is too selective etcetera, etcetera**

'bag' is problematic as it's partially synonymous with multiset (and colony is not one of those) and partially because it's vague - it doesn't describe how the container works. I have not come up with an alternate name that fits as well, nor has anyone as yet. 'colony' is an intuitive name if you understand the container, and allows for easy conveyance of how it works. The suggestion that the use of the word is selective in terms of it's meaning, is also true for vector, set, 'bag', and many other C++ names. Largely, misgivings appear to stem from unfamiliarity with the term. Non-verbose alternatives which don't lead to incorrect assumptions are always appreciated.

It is true that 'colony' says more about it's structure than it's usage, however I think it is valuable to understand how colony works, in this case. If someone were to erase an element, then insert a new object, they might be surprised to discover that pointers to the erased element may now point to the newly-inserted one, unless they had a basic understanding of how the container works. Also if they inserted an element and subsequently discover the element is at index four in the iterative sequence rather than at the back of the colony, this could be confusing to a newcomer, however it is easily explained via the metaphorical explanation in the introduction of this document. Personally I would rather have 'suitcase' than bag - it's only slightly more silly, and implies location stability, which is what a bag container typically lacks.

2. **"Unordered and no associative lookup, so this only supports use cases where you're going to do something to every element."**

As noted the container is designed for highly object-oriented situations where you have many elements in different containers referring to many other elements in other containers. This can be done with pointers or iterators in colony (insert returns an iterator which can be dereferenced to get a pointer, pointers can be converted into iterators with the supplied functions (for erase etc)) and because pointers/iterators stay stable regardless of insertion/erasure, this usage is

unproblematic. You could say the pointer is equivalent to a key in this case (but without the overhead). That is the first access pattern, the second is straight iteration over the container, as you say. Secondly, the container does have (typically better than O(N)) advance/next/prev implementations, so elements can be skipped.

3. **"Do we really need the skipfield_type template argument?"**

   If it is possible to relegate this to a constructor argument (I'm not sure how exactly) I would welcome it, but this argument currently promotes use of the container in heavily-constrained memory environments, and in high-performance small-N collections. Unfortunately it also means `operator =` and some other functions won't work between colonies of the same type but differing skipfield types. See more explanation in V. Technical Specifications. This is something I am flexible on, as a singular skipfield type will cover the majority of scenarios.

4. **"Prove this is not an allocator"**

   I'm not really sure how to answer this, as I don't see the resemblance, unless you count maps, vectors etc as being allocators also. The only aspect of it which resembles what an allocator might do, is the memory re-use mechanism. However, this could not be used by any container which was not set up specifically to iterate with a skipfield (or which had a non-contiguous iterative mechanism), and so is reasonably specific. I don't claim that the way colony re-uses memory is anything unique - slot_map also performs this operation - but it *is necessary to the structure as a whole*, and will not function without it, just as it will not function without the jump-counting skipfield pattern or a block-based memory allocation pattern. Each of these is in service of colony's core goals: fast insertion and erasure, reasonable iteration speed and a lack of invalidation of pointers to non-erased elements upon insertion and erasure.

Appendix E - Open Questions from the author

1. Should "shrink_to_fit()" follow the semantics of vector or deque? Deque shrink_to_fit() is geared more towards freeing unused memory blocks, not invalidating existing pointers-to-elements, whereas vector shrink_to_fit() consolidates all elements into a memory block which exactly fits size(). If we follow the semantics of vector, a secondary "free_unused_memory()" function will be necessary. If we follow deque semantics, we lose optimization strategies for consolidating the colony (see code for shrink_to_fit() in current implementation). Currently this proposal uses deque semantics.
2. Should reserve() be implemented at all? Since there is no reallocation upon instantiation of a new memory block, insert is not a costly operation when size == capacity. The main advantage I can see for retaining it is for high-performance/game programming, where allocation operations can be moved to non-timing-critical code.
3. Name? I can't come up with a better one. I am more or less resigned to the fact that the eventual name will be 'bag', but I just want to go on record as saying: 'That's terrible. You should not do that.' for the reasons stated above.

Appendix F - Paper revision history

- R4: Addition of revision history and review feedback appendices. General rewording. Update of benchmarks to v4 of colony, using max 1000000 N for most benchmarks, and using GCC 7.1 as compiler on a Haswell-core machine. Previous benchmarks also still available at external links. Expansion of initial metaphorical explanation. Cutting of some dead wood. Addition of some more dead wood. Reversion to HTML, benchmarks moved to external URL, based on feedback. Change of font to Times New Roman based on looking at what other papers were using, though I did briefly consider Comic Sans. Change to insert specifications.
- R3: Jonathan Wakely's extensive technical critique has been actioned on, in both documentation and the reference implementation. "Be clearer about what operations this supports, early in the paper." - done (V. Technical Specifications). "Be clear about the O() time of each operation, early in the paper." - done (for main operations V. Technical Specifications). Responses to some other feedbacks included in the foreword.
- R2: Rewording.